

Old Timer Fights On!

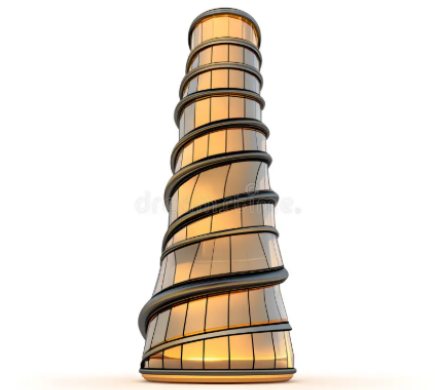
Table of Contents:

- Game Overview
- Core Mechanics
- Visual & Audio Style
- Technical Architecture
- Development Milestones
- Risk Assessment

Game Overview

Overview: As the title of the document suggests, the title of this game will be *Old Timer Fight's On!* The genre for this game will be a 3rd person action roguelike. Currently, the target platform would be primarily on Unity. Devices that would play 3d games would be more than enough for this game.

Concept: *Old Timer Fights On!* Begins with the player character, *Old Timer*, waking up outside of the tower *Slanted Memories*. Dastardly people have taken over *Slanted Memories*, so *Old Timer* sets out to restore peace within the tower. No matter how many times *Old Timer* falls, he will come back stronger!



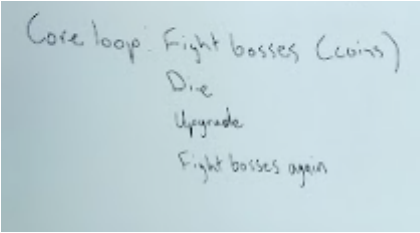
Target Audience: Typical gamers, catering to those who have played other action roguelikes (like Dark Souls).

Core Mechanics

Player Controls: Basic 4-way movement, aiming/shooting with a gun, jumping. Simple controls to navigate menus.

Primary Mechanics: Just like any boss in a video game, the end goal of the game is for the player to beat all the bosses. If a player dies to a boss, they need to restart from the beginning, but can buy upgrades using coins to make themselves stronger.

Game Loop:



```
graph TD; A[Core loop: Fight bosses (coins)] --> B[Die]; B --> C[Upgrade]; C --> D[Fight bosses again]; D --> A;
```

Core loop: Fight bosses (coins)
Die
Upgrade
Fight bosses again

Visual & Audio Style

Art Direction & References: The game's art style will be something a bit cartoonish. It would be similar to the style of *No More Heroes*. Where most characters are average humans, but make exaggerated movements, powers, and/or weapons.





UI/UX Considerations: The plan for the UI is to keep it as simple as possible. Displaying the player's health, the boss's health, and the ammo count for the player's gun would be enough. It would be similar to *Kirby and the Forgotten Land*, where the UI is minimal.

Sound Design Approach: The game plans to have typical sounds during actions (for both players and enemies). For actions that will happen often (shooting, jumping, walking, getting hurt), some thought will need to go into preventing players from feeling annoyed/overwhelmed by them.

Level Design: One aspect that should be clear before moving on is that the level designs of this game will mainly be arenas. The only different area would be the starting area (outside the tower).

Technical Architecture

Pattern Implementations: (Add in photo of test discussing implementations) The two not planned would be the Command and Spatial Partition Patterns.

Key Systems: A combat system where the player has to learn the boss's attacks and attack at the right time. An upgrade system that allows the player to be



stronger, and a weapon system that allows players to change aspects of their gun (effects, capacity, damage).

Third-Party Assets/Plugins: Most assets of *Old Timer Fights On!* Will come from the Unity Asset Store. Characters, areas, props, and effects will all be from there. The sounds used for the game will come from *Freesound.com*.

Development Milestones

Since this game is only being created by one person (with additional help from a professor), the current “end-goal” is to have three fightable bosses. Alongside that, some upgrades/attachments for the player’s gun are also considered. This is the current goal if other classes don’t affect time.

Timeline:

- Week 1 (3/30 - 4/5)
 - Finish up the Game Development Document
- Week 2 (4/6 - 4/12)
 - Download assets to use for the game
 - Begin implementations of simple patterns
- Week 3 (4/13 - 4/19)
 - Continue implementations of patterns
 - Create scenes for each area
- Week 4 (4/20 - 4/26)

- Continue implementations of patterns
- Continue working on scenes
- Week 5 (4/27 - 5/3)
 - Last-minute checkup
 - Present/turn in

Risk Assessment

Technical Challenges: The main challenge in getting this game to be playable heavily depends on finding proper assets to use.

Mitigation Strategies: Reuse assets whenever possible. Mainly reuse props and effects when possible.



Features to Cut: Limit the number of unique bosses to fight. To artificially increase numbers, create variations of bosses. Reuse scenes if necessary.